



Orientação a Objetos em C++

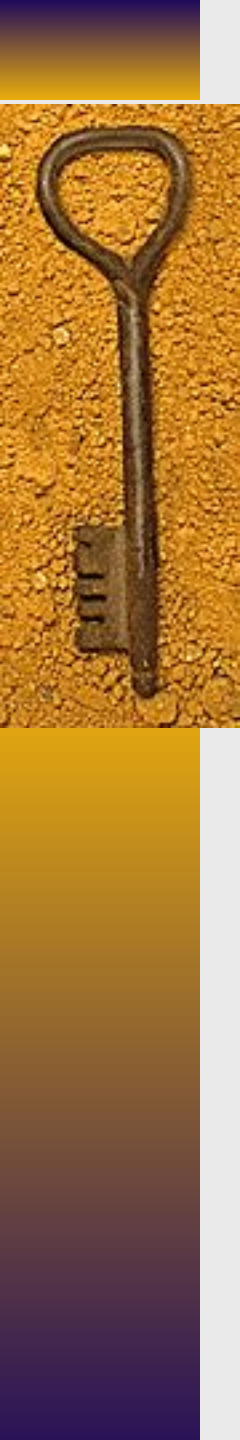
Prof. Dr. Valter Vieira de Camargo

POO - 2026/1



Namespaces e Using

- ♦ Como evitar toda vez digitar “**std::string**” ? (ou coisas similares)
- ♦ O “using” é um tipo de atalho
- ♦ Quando você digita “**using namespace std**” você traz TUDO de dentro da biblioteca std para o escopo do seu código e não precisa mais dizer que string é uma keyword que está em **std**.
- ♦ Então, vc pode usar somente a palavra “string” para criar variáveis
- ♦ ex → **string nome**
- ♦ Mas cuidado, é mais seguro, ser mais específico e trazer só o que irá usar “using std::string”
- ♦ Quando você faz using namespace std você traz MUITA coisa



Namespaces e Using

- ◆ Quando você traz tudo da biblioteca `std`, pode causar conflitos de nomes
- ◆ Por exemplo:
 - criar uma variável com o nome *cout*... não será possível pois *cout* também é uma keyword .
 - `int cout = 10`
- ◆ O ideal é importar somente o que precisa:
 - `using std::cout`
 - `using std::string`
- ◆ Isso é melhor do que importar tudo.
- ◆ Faça sempre deste jeito

O que tem dentro de Std ?



Categoria	Exemplos	Para que serve
Entrada/Saída	<code>std::cout</code> , <code>std::cin</code> , <code>std::endl</code>	Comunicação com usuário
Strings	<code>std::string</code>	Manipulação de texto
Containers	<code>std::vector</code> , <code>std::list</code> , <code>std::map</code> , <code>std::set</code>	Estruturas de dados
Algoritmos	<code>std::sort</code> , <code>std::find</code> , <code>std::reverse</code>	Operações sobre dados
Iteradores	<code>std::begin</code> , <code>std::end</code>	Navegar em containers
Funções matemáticas	<code>std::sqrt</code> , <code>std::pow</code> , <code>std::abs</code>	Cálculos matemáticos
Utilidades	<code>std::pair</code> , <code>std::tuple</code> , <code>std::swap</code>	Estruturas auxiliares
Tempo	<code>std::chrono</code>	Medição de tempo
Manipulação de arquivos	<code>std::ifstream</code> , <code>std::ofstream</code>	Leitura/escrita em arquivos
Exceções	<code>std::exception</code> , <code>std::runtime_error</code>	Tratamento de erros
Ponteiros inteligentes	<code>std::unique_ptr</code> , <code>std::shared_ptr</code>	Gerenciamento de memória
Funções e lambdas	<code>std::function</code>	Funções como objetos
Concorrência	<code>std::thread</code> , <code>std::mutex</code>	Programação paralela
Random	<code>std::random_device</code> , <code>std::mt19937</code>	Números aleatórios
Containers adaptadores	<code>std::stack</code> , <code>std::queue</code> ,	Estruturas específicas



“using” para apelidos (alias)

Outra forma de usar “using” é para dar apelidos para tipos que já existem.

Útil em alguns casos ... mas tem que usar com cautela

Por exemplo:

- ◆ `using inteiro = int;`
- ◆ `using ListaDeNomes = std::vector<std::string>;`
- ◆ `inteiro idade = 30;`
- ◆ `ListaDeNomes nomes = {"Ana", "Bruno"};`

“using” para apelidos (alias)

Exemplo - Lista de Produtos

```
#include <iostream>
#include <vector>
#include "Produto.h"
```

```
using std::vector;
```

```
using ListaProdutos = vector<Produto>;
```

//Apelido criado

```
int main() {
```

```
    ListaProdutos produtos;
```

```
    Produto p1(1, "Teclado", 100.0);
```

```
    Produto p2(2, "Mouse", 50.0);
```

```
    produtos.push_back(p1);
```

```
    produtos.push_back(p2);
```

```
    for (int i = 0; i < produtos.size(); i++) {
```

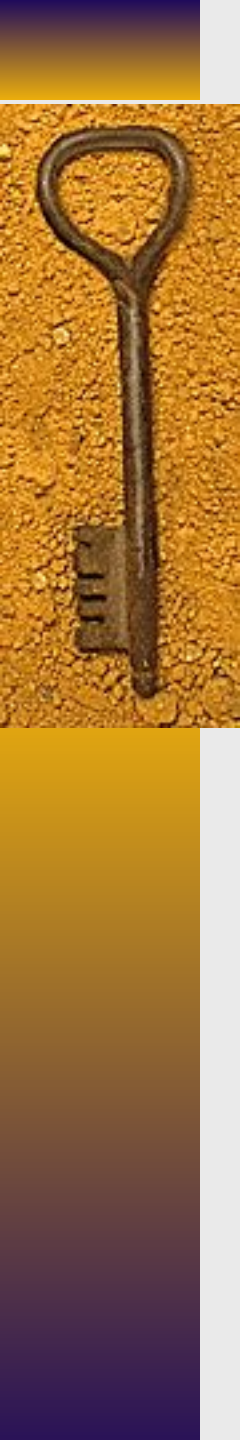
```
        produtos[i].imprimir();
```

```
    }
```

```
    return 0;
```

```
}
```





Include x Using

- ◆ Cuidado com a diferença entre **#include** e **using**
 - `#include <string>` → o pré-processador inclui no seu código antes da compilação
 - `#include <string>` traz para dentro do seu código todas as definições de `String` (a classe `String`, funções relacionadas com `string`, tipos auxiliares, etc)
 - mas ... tudo continua dentro de `std` → `std::string`, `std::getline`
- ◆ O `#include` é obrigatório, o `using` não.
- ◆ O `using` remove a necessidade de escrever o escopo daquilo que estou usando.
- ◆ O `using` “traz” tudo para dentro do seu código - é como se ele tirasse do escopo original.

Const

- ◆ Quando usado no parâmetro de um método, garante que aquele parâmetro não será modificado dentro do corpo do método
- ◆ A verificação é em tempo de compilação
- ◆ Suponha que você queira atribuir um novo valor a um atributo, por exemplo uma nova idade a um funcionário

```
Funcionario valter (1, "Valter", 50, endValter);  
valter.setIdade(51);
```

- ◆ Você tem certeza de que o valor 51 realmente será atribuído à idade ?
- ◆ Lembre-se, podem ser pessoas diferentes que definiram a interface e o corpo do método

```
void Funcionario::setIdade(int i) {  
    if (i >= 0) {  
        i = 200 *****  
        idade = i;  
    } else {  
        cout << "Idade invalida!" << endl;  
    }  
}
```




Const

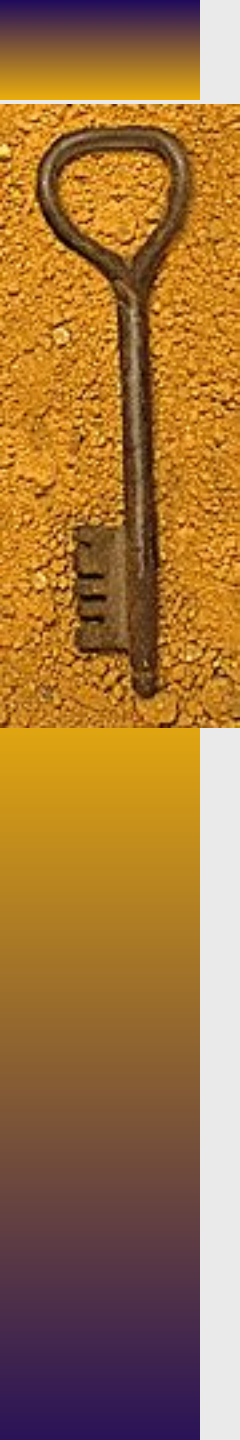
- ◆ Para que o desenvolvedor da interface (.h) tenha certeza de que a implementação não mudará o valor do parâmetro, deve-se utilizar o **const**

No .h → `void setIdade(const int idade);`

No .cpp

```
void Funcionario::setIdade( const int idade) {  
    idade = 10    // dá erro de compilação !!  
    this->idade = idade;  
  
}
```

- ◆ Então, todos os sets, devem seguir esse padrão



Const

- ◆ Isto também se aplica para métodos set que recebam como parâmetro um objeto.
- ◆ A diferença é que a passagem de objetos como parâmetros é melhor ser feita por referência (&) para **economizar espaço de memória**
- ◆ Entretanto, usando “const” também não terá risco de se alterar o valor da variável original de qq forma

```
void Funcionario::setEndereco (const Endereco& end) {  
    end.setRua("nova rua"); //erro de compilação  
    this->endereco = end;  
}
```



Const em métodos get()

- ♦ E se quisermos também ter certeza de que os métodos get() não faça alterações em nenhum dos atributos do objeto atual ?
- ♦ Basta fazer:
No .h → `int getIdade() const;`
No .cpp → `int Funcionario::getIdade() const {`
 `return idade;`
 `this->qqAtributo = 10 //Erro de compilação`
 `}`

Dessa forma, sempre devemos declarar o `const` para métodos `get()`

Funcionario.h com const

```
#ifndef FUNCIONARIO_H  
#define FUNCIONARIO_H
```

```
#include "Endereco.h"
```

```
using std::string;
```

```
class Funcionario {  
private:
```

```
    int codigo;  
    string nome;  
    int idade;  
    Endereco endereco;
```

```
public:
```

```
    Funcionario(const int c, const string n, int i, const Endereco& endereco); // Construtor  
    Funcionario (const string, int);  
    Funcionario();
```

```
    void apresentar(); // Método para exibir informações
```

```
    void setCodigo(const int);  
    int getCodigo() const;
```

```
    void setNome(const string);  
    string getNome() const;
```

```
    void setIdade(const int);  
    int getIdade() const;
```

```
    void setEndereco(const Endereco&);  
    Endereco getEndereco() const;
```

```
};  
#endif
```



Funcionario.cpp com const

using namespace std;

```
Funcionario::Funcionario(const int c, const string n, const int i, const Endereco& end)
    : codigo(c), nome(n), idade(i), endereco(end) {}
```

...

```
void Funcionario::setEndereco (const Endereco& end){
```

```
    //end.setRua("nova rua"); //erro de compilação
```

```
    this->endereco = end;
}
```

```
Endereco Funcionario::getEndereco() const{
    return this->endereco;
}
```

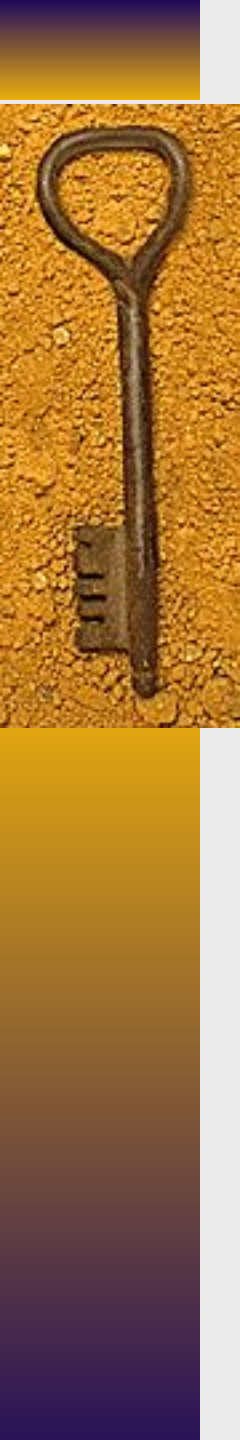
```
void Funcionario::setCodigo(const int codigo) {
    this->codigo = codigo;
}
```

```
int Funcionario::getCodigo() const {
    return codigo;
}
```

```
void Funcionario::setNome(const string nome) {
    this->nome = nome;
}
```

```
std::string Funcionario::getNome() const {
    return nome;
}
```

...



Endereco.h com const

```
using std::string;
```

```
class Endereco {  
private:
```

```
    string rua;  
    string cidade;  
    string estado;  
    string cep;
```

```
public:
```

```
    Endereco();  
    Endereco(const string rua, const string cidade, const string estado, const string cep);  
    void exibirEndereco();
```

```
    void setRua(const string);  
    void setCidade(const string);  
    void setEstado(const string);  
    void setCep(const string);
```

```
    string getRua() const;  
    string getCidade() const;  
    string getEstado() const;  
    string getCep() const;
```

```
};  
#endif
```



Endereco.cpp com const

```
#include "Endereco.h"
#include <iostream>
```

```
using std::string;
```

```
Endereco::Endereco() {}
```

```
Endereco::Endereco(const string rua, const string cidade, const string estado, const string cep)
    : rua(rua), cidade(cidade), estado(estado), cep(cep) {}
```

```
void Endereco::exibirEndereco() {
```

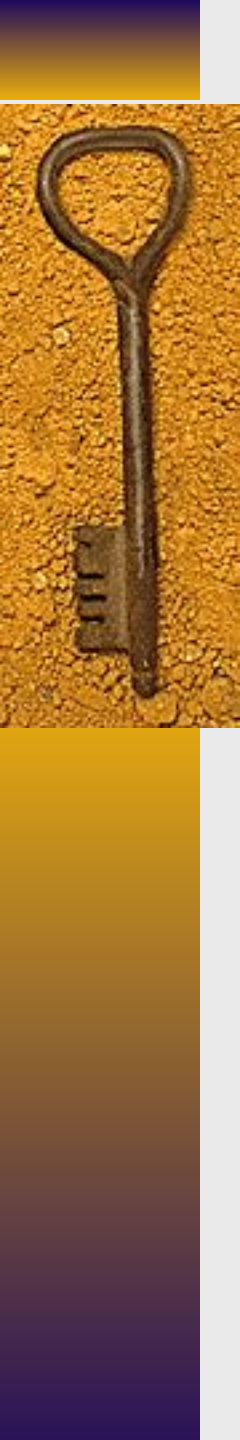
```
    std::cout << "Endereco: " << rua << ", " << cidade << " - " << estado << ", " << cep << endl;
}
```

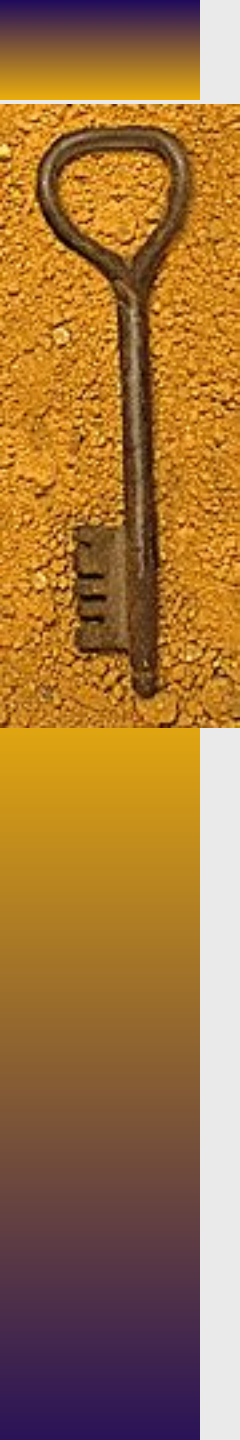
```
void Endereco::setRua(const string novaRua){
    this->rua = novaRua;
}
```

```
string Endereco::getRua() const{
    return this->rua;
}
```

```
void Endereco::setCidade(const string cidade){
    this->cidade = cidade;
}
```

```
string Endereco::getCidade() const{
    return this->cidade;
}
```





Enumerações

- ♦ São estruturas que agrupam conjuntos de valores, normalmente lista de valores fixos
- ♦ Enumerações não são classes
- ♦ exemplos:
 - dias da semana
 - meses do ano
 - cargos em uma empresa
- ♦ Enumerações não possuem atributos e/ou métodos
- ♦ Exemplo que iremos fazer:
 - Imagina que Funcionario agora pode ter **cargos**

Exemplo Enumeração

```
#include <iostream>
#include <string>
```

```
enum class Cargo {
    CEO,
    CTO,
    Secretaria,
    Presidente,
    Estagiario
};
```

```
// Função auxiliar para converter Cargo em string (opcional, mas útil)
std::string cargoToString(Cargo c) {
    switch (c) {
        case Cargo::CEO: return "CEO";
        case Cargo::CTO: return "CTO";
        case Cargo::Secretaria: return "Secretaria";
        case Cargo::Presidente: return "Presidente";
        case Cargo::Estagiario: return "Estagiario";
        default: return "Cargo desconhecido";
    }
}
```

```
int main() {
    Cargo cargo = Cargo::CTO;

    std::cout << "Cargo selecionado: " << cargoToString(cargo) << std::endl;

    return 0;
}
```

Embora não fique explícito, os itens da enumeração são constantes nomeadas e possuem um índice, começando em 0. Isto é, CEO é 0, CTO é 1 ...

Se você passar um nro inteiro como argumento para o enum, ele será convertido para um dos valores do enum

Enumerações dentro de classes

```
#include <iostream>
#include <string>
using namespace std;
```

```
class Cargo {

public:
    enum Valor {
```

CORREÇÃO !!!!

APÓS MELHOR ANÁLISE, CONSTATEI QUE
ENCAPSULAR ENUMS DENTRO DE CLASSES
ACABA COMPLICANDO MAIS.

A MELHOR PRÁTICA EM C++ É DEIXAR OS
ENUMS EM ARQUIVOS SEPARADOS, MAS
NÃO NECESSARIAMENTE EM NOVAS
CLASSES

ASSIM, VOCÊS DEVEM SEPARAR O ENUM EM
ARQUIVOS SÓ PARA ELES, MAS NÃO
PRECISA ENCAPSULAR ELES EM CLASSES

Neste caso, estamos criando uma classe
Wrapper, que empacote o enum dentro
dela.

Isso permite criar métodos que manipulam
o enum, como se ele fosse uma classe

Inicializa com o “valor padrão” do enum,
que é o primeiro item ... no caso CEO

toString → método para imprimir em
String os valores

Enums em Arquivos Separados

```
#ifndef STATUS_PEDIDO_H  
#define STATUS_PEDIDO_H
```

STATUS_PEDIDO.H

```
#include <string>
```

```
enum class StatusPedido {  
    ABERTO,  
    EM_PREPARO,  
    FINALIZADO,  
    CANCELADO  
};
```

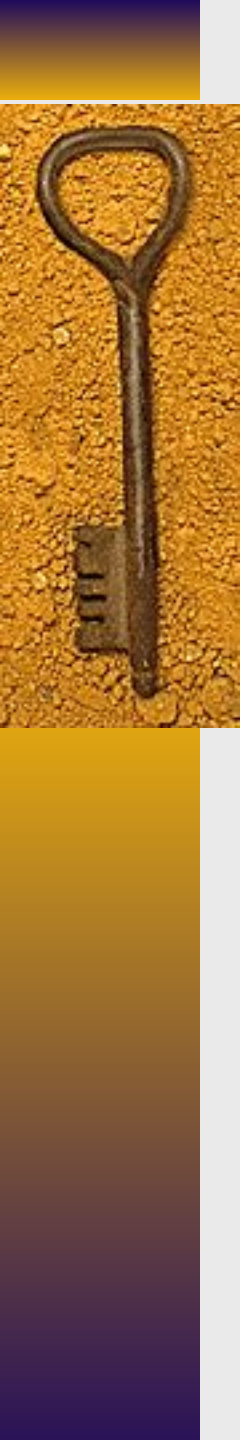
```
std::string toString(StatusPedido  
status);
```

```
#endif
```

STATUS_PEDIDO.CPP

```
#include "StatusPedido.h"
```

```
std::string toString(StatusPedido status) {  
    switch (status) {  
        case StatusPedido::ABERTO: return "Aberto";  
        case StatusPedido::EM_PREPARO: return "Em preparo";  
        case StatusPedido::FINALIZADO: return "Finalizado";  
        case StatusPedido::CANCELADO: return "Cancelado";  
    }  
    return "";  
}
```



Enumerações (Funcionario.h)

```
class Funcionario {  
private:  
    int codigo;  
    string nome;  
    int idade;  
    Endereco endereco;  
    Cargo cargo;
```

```
public:  
    Funcionario(int c, string n, int i, Endereco& endereco, const Cargo& cargo);  
    Funcionario(int c, string n, int i, Endereco& endereco);  
    Funcionario (string, int);  
    Funcionario();  
    ...  
  
    void setCargo(const Cargo& cargo);  
    Cargo getCargo();
```



Enumerações (Funcionario.cpp)

```
...
using namespace std;

Funcionario::Funcionario(int c, string n, int i, Endereco& end, const Cargo& car)
    : codigo(c), nome(n), idade(i), endereco(end), cargo(car) {}

Funcionario::Funcionario(int c, string n, int i, Endereco& end)
    : codigo(c), nome(n), idade(i), endereco(end) {}

Funcionario::Funcionario(string n, int i)
    : codigo(0), nome(n), idade(i) {}

void Funcionario::apresentar() {
    cout << "---- PRINTANDO DADOS ----" << endl;

    cout << "Codigo: " << codigo << endl;
    cout << "Nome: " << nome << endl;
    cout << "Idade: " << idade << endl;
    cout << "Cargo: " << cargo.toString() << endl;

    endereco.exibirEndereco();

    cout << "-----" << endl;
}

void Funcionario::setCargo(const Cargo& c) {
    this->cargo = c;
}

Cargo Funcionario::getCargo() {
    return this->cargo;
}
...
```




Enumerações (Main)

```
int main() {
```

```
    Endereco endValter ("Rua das Flores", "São Paulo", "SP", "01234-567");  
    Endereco endArthur ("Alameda dos Crisântemos", "São Carlos", "SP",  
"14940-000");
```

```
    Funcionario valter (1, "Valter", 50, endValter, Cargo::CTO);  
    Funcionario arthur (2, "Arthur", 14, endArthur);
```

```
    arthur.setCargo(Cargo::Estagiario);
```

```
    valter.apresentar();  
    cout << "O cargo do Arthur é : " << arthur.getCargo().toString() << endl;
```

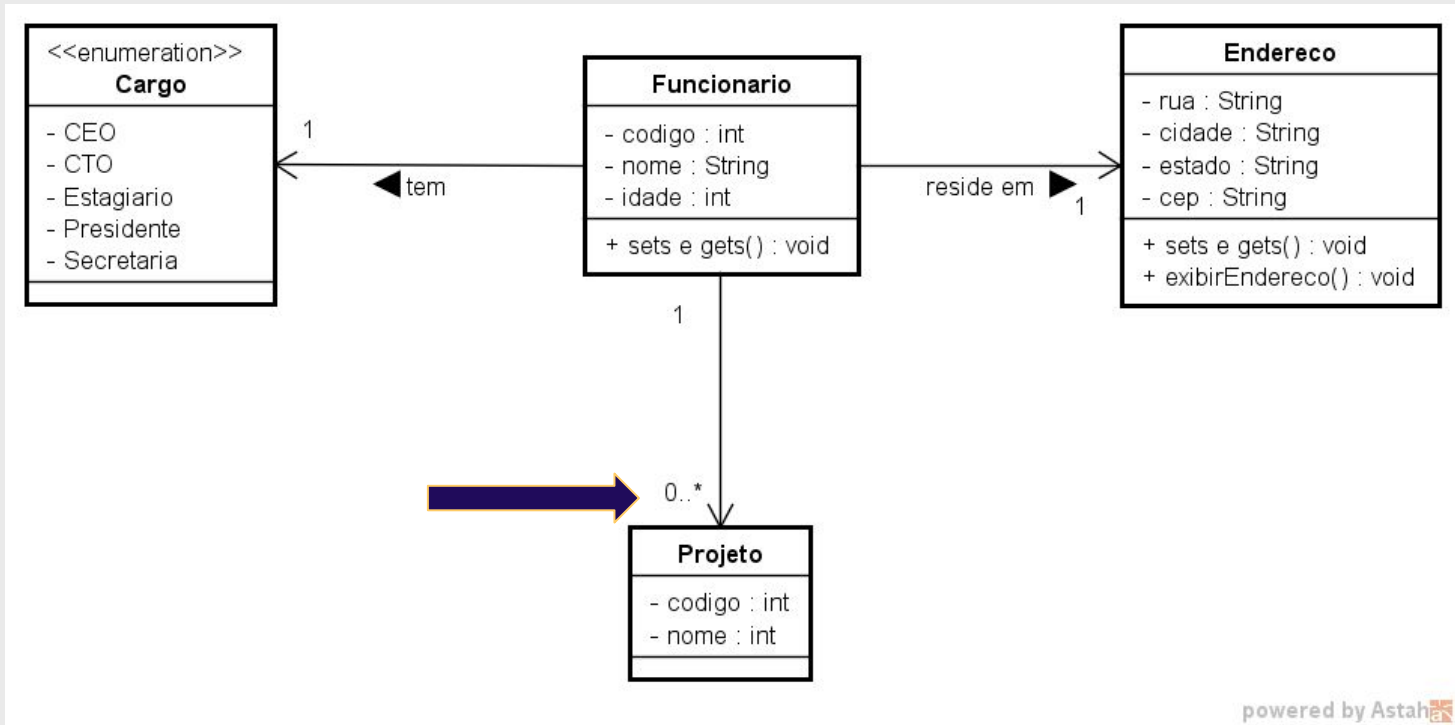


Entenda bem isso !

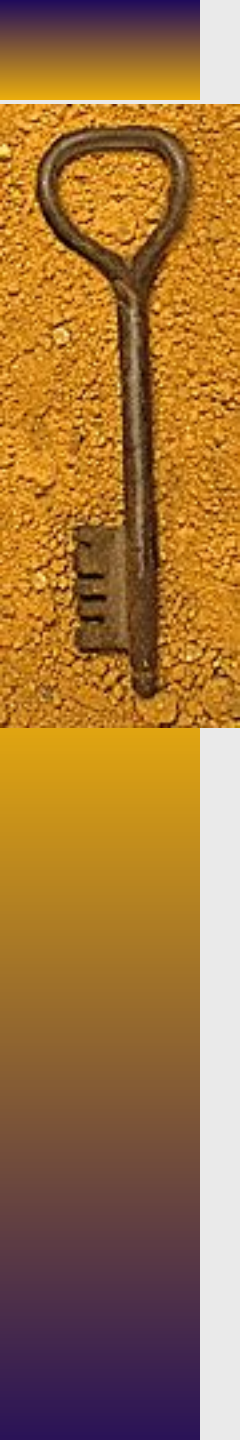


Relacionamentos 1 para n

Relacionamentos 1 ..n



- ◆ Funcionário pode estar envolvido em nenhum ou vários projetos.
- ◆ Como implementar ?
 - Precisamos armazenar, dentro de um objeto funcionário, todos os objetos “projeto” que esse funcionário atua



Relacionamentos 1 ..n

- ♦ Pontos importantes
 - Sabe-se de antemão em quantos projetos um funcionário pode trabalhar ? não !
 - Qual estrutura de dados usar ?
 - Lista encadeada → só em ED
 - Array → estrutura que deve ser criada já com um tamanho fixo
 - Classe **vector** da biblioteca std
 - estrutura que aloca memória e desloca dinamicamente,
 - usa um bloco contínuo de memória

Array x Vetores

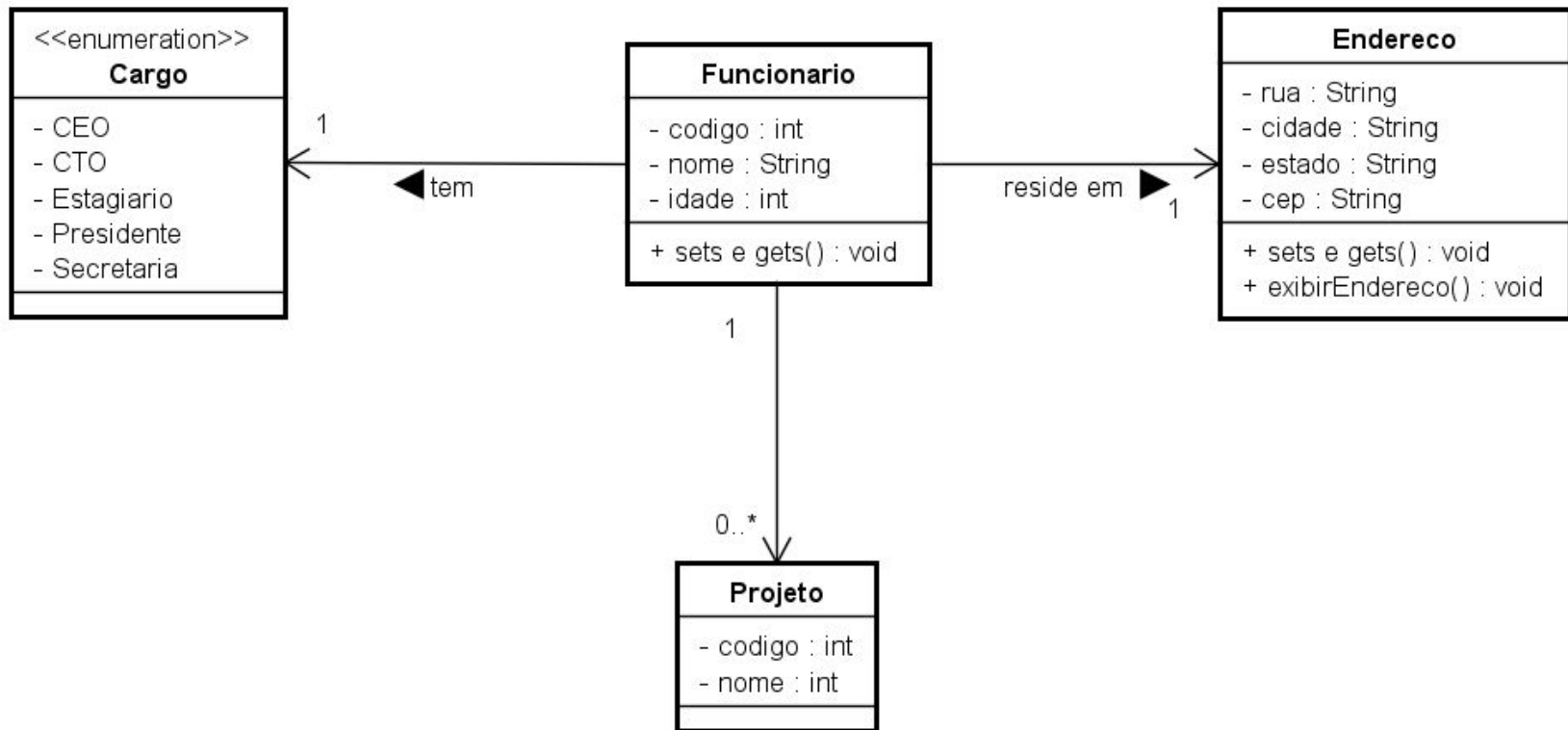
array vs vector

Característica	array (C/C+ tradicional)	vector (std::vector da STL)
Tamanho	Fixo no momento da criação	Dinâmico (pode crescer ou diminuir automaticamente)
Redimensionamento	Não é possível (sem criar novo array)	Sim, automático com <i>push_back</i> , <i>resize</i> , etc.
Alocação de memória	Contígua e fixa	Contígua e gerenciada dinamicamente
Gerenciamento de memória	Manual (em arrays dinâmicos com <i>new/delete</i>)	Automático (RAII – libera ao sair do escopo)
Métodos integrados	Não possui métodos integrados	Possui muitos métodos (itens: <i>size()</i> , <i>clear()</i> , <i>push_back()</i> etc.
Facilidade de uso	Mais difícil (sem métodos auxiliares)	Mais fácil e seguro de usar
Acesso a elementos	Rápido ($O(1)$) via 'índice	Pode usar <i>at()</i> que verifica limites
Verificação de limites (bounds)	Não há (acesso fora dos limites pode causar erro)	Automatizada (especialmente no final com <i>push_back</i>)
Inserção / Remoção	Manual e ineficiente (copiar elementos)	Levemente mais pesado (mas muito otimizado)
Parte de qual biblioteca?	C++ nativo (C-style arrays)	Biblioteca Padrão C++ (STL)

Métodos da Classe Vector

Método	Descrição	Exemplo de uso
<code>push_back()</code>	Adiciona um elemento ao final do vetor.	<code>v.push_back(10);</code>
<code>pop_back()</code>	Remove o último elemento do vetor.	<code>v.pop_back();</code>
<code>size()</code>	Retorna o número de elementos.	<code>v.size();</code>
<code>empty()</code>	Retorna <code>true</code> se o vetor estiver vazio.	<code>if(v.empty()) { ... }</code>
<code>clear()</code>	Remove todos os elementos.	<code>v.clear();</code>
<code>at(i)</code>	Acessa o elemento na posição <code>i</code> com verificação de limites.	<code>v.at(2);</code>
<code>operator[]</code>	Acessa o elemento na posição <code>i</code> sem verificação de limites.	<code>v[2];</code>
<code>insert(pos, val)</code>	Insere um valor na posição especificada.	<code>v.insert(v.begin() + 1, 5);</code>
<code>erase(pos)</code>	Remove o elemento na posição especificada.	<code>v.erase(v.begin() + 1);</code>
<code>begin()</code> / <code>end()</code>	Retorna iteradores para o início/fim do vetor.	<code>for(auto it = v.begin(); it != v.end(); ++it)</code>
<code>resize(n)</code>	Redimensiona o vetor para conter <code>n</code> elementos.	<code>v.resize(10);</code>
<code>capacity()</code>	Retorna a capacidade atual do vetor (quantos elementos cabem).	<code>v.capacity();</code>
<code>reserve(n)</code>	Garante capacidade para pelo menos <code>n</code> elementos (evita realocações).	<code>v.reserve(100);</code>
<code>shrink_to_fit()</code>	Reduz a capacidade para caber exatamente os elementos atuais.	<code>v.shrink_to_fit();</code>
<code>front()</code> / <code>back()</code>	Retorna o primeiro/último elemento.	<code>v.front();</code> , <code>v.back();</code>

Relacionamentos 1 ..n



Inicialmente, criar a classe Projeto



Classe Projeto.h

```
...  
using namespace std;  
  
class Projeto {  
private:  
    int codigo;  
    string nome;  
  
public:  
    // Construtores  
    Projeto(const int c, const string& n); // Construtor com parâmetros  
    Projeto(); // Construtor padrão  
  
    // Métodos get e set  
    void setCodigo(const int c);  
    int getCodigo() const;  
  
    void setNome(const string& n);  
    string getNome() const;  
  
    // Método para imprimir os dados  
    void imprimir() const;  
};
```

Projeto.cpp

```
#include "Projeto.h"
```

```
// Construtor com parâmetros
```

```
Projeto::Projeto(const int c, const string& n) : codigo(c), nome(n) {}
```

```
// Construtor padrão
```

```
Projeto::Projeto() : codigo(0), nome("") {}
```

```
void Projeto::setCodigo(const int c) {  
    codigo = c;  
}
```

```
void Projeto::setNome(const string& n) {  
    nome = n;  
}
```

```
int Projeto::getCodigo() const {  
    return codigo;  
}
```

```
string Projeto::getNome() const {  
    return nome;  
}
```

```
void Projeto::imprimir() const {  
    cout << "Projeto Código: " << codigo << ", Nome: " << nome << endl;  
}
```

Classe Funcionario.h

```
#include <vector>
```

```
class Funcionario {  
private:
```

```
...
```

```
vector<Projeto> projetos; // Isto é um vetor de objetos do tipo Projeto
```

```
public:
```

```
...
```

```
// Métodos relacionados a projetos
```

```
void setProjetos(const vector<Projeto>&);
```

```
vector<Projeto> getProjetos() const;
```

```
Projeto getProjetoPorCodigo(const int codigo) const;
```

```
void adicionarProjeto(const Projeto& p); // Método para adicionar um projeto
```

```
};
```



Classe Funcionario.cpp



```
void Funcionario::apresentar() const {
    cout << "--- PRINTANDO DADOS ---" << endl;

    cout << "Codigo: " << codigo << endl;
    cout << "Nome: " << nome << endl;
    cout << "Idade: " << idade << endl;
    cout << "Cargo: " << cargo.toString() << endl;

    endereco.exibirEndereco();

    cout << "Projetos do funcionário:" << endl;

    for (const Projeto& p : projetos) {
        p.imprimir();
    }

    cout << "-----" << endl;
}
```

Classe Funcionario.cpp



```
Projeto Funcionario::getProjetoPorCodigo(const int codigo)
const {
    for (const Projeto& p : projetos) {
        if (p.getCodigo() == codigo) {
            return p;
        }
    }
    throw std::runtime_error("Projeto com o código informado não
    encontrado.");
}

void Funcionario::adicionarProjeto(const Projeto& p) {

    projetos.push_back(p);

}
```



Exercício

Exercício – Sistema de Pedidos de Restaurante

Implemente, em C++, um sistema orientado a objetos simplificado para gerenciamento de pedidos em um restaurante.

Um **Pedido** pode conter vários **itens**, e cada item representa um produto solicitado pelo cliente (por exemplo: pizza, bebida, sobremesa, etc.).

Requisitos

1. Modele o sistema utilizando conceitos de orientação a objetos:
 - Crie uma classe que represente um **Item do Pedido** (nome do produto, quantidade e preço unitário).
 - Crie uma classe que represente um **Pedido (ID, descrição, data, valor e o conjunto de itens daquele pedido)**.
2. Cada pedido deve possuir:
 - um identificador
 - uma situação (por exemplo: aberto, em preparo, finalizado, cancelado)
 - uma coleção de itens (um pedido pode ter vários itens)
3. Modele a situação do pedido utilizando um tipo da linguagem que represente um conjunto fixo de valores possíveis.
4. A classe Pedido deve oferecer operações como:
 - adicionar um novo item ao pedido
 - calcular o valor total do pedido
 - listar os itens do pedido
 - consultar informações do pedido (como identificador e situação)
5. Considere boas práticas de programação:
 - evite redundâncias ao utilizar elementos da biblioteca padrão da linguagem
 - métodos que apenas consultam dados do objeto devem ser projetados de forma adequada a esse comportamento
6. No programa principal:
 - crie pelo menos dois pedidos
 - adicione diferentes itens a cada pedido
 - altere a situação de pelo menos um pedido
 - exiba na tela os dados dos pedidos, incluindo seus itens e o valor total

Observação:

Organize seu código de forma clara e coerente, separando responsabilidades entre as classes e utilizando adequadamente os recursos da linguagem para tornar o código mais legível e reutilizável.

